

Inventory Structure — As Built

NextBook ERP

5 August 2026

Reference for the multi-business inventory core. Covers supermarket, grocery, manufacturing, pharmacy, automobile, book, garment, jewellery, mobile, carpet, computer and electric.

Design goal: onboarding a new business type must require **seed data only** — no new tables, no new columns.

Status: **built and applied**. Migrations are live on the development database and covered by [tests/Feature/Inventory/ItemCrudTest.php](#).

1. The four levels

Level	Table	Answers	Carries
Item	items	What product line is it?	name, category, accounts, policy
Variant	item_variants	Which one does the customer choose?	SKU, barcode, price, min stock
Lot	stock_lots	Which delivery did it arrive in?	batch, expiry, MRP, lot cost
Piece	stock_pieces	Which exact physical object?	serial/IMEI, dimensions, actual cost

[stock_balances](#) remains the quantity ledger and points at variant and lot.

Decision rule

Ask in order; first "yes" wins.

1. Does the **customer choose** between them? → **Variant attribute**
2. Is it the same for **every unit** of the product? → **Item field**
3. Does it arrive **with a delivery**? → **Lot**
4. Is it unique to **one physical object**? → **Piece**

Rules enforced in code

- **Every item has at least one variant.** Items with no choices get a default variant with empty attributes, so [variant_id](#) is never null downstream and barcode scanning and price lookup have a single path. Enforced in [ItemStoreRequest::prepareForValidation\(\)](#) and [ItemVariantService::normalise\(\)](#).
- **Exactly one default variant.** Enforced in the request and again in the service.
- **Lots are optional.** No batch and no expiry → no lot row.
- **Pieces are optional.** Only serial/unique-piece items create them.
- For serial-tracked items: [stock_balances.quantity == count\(stock_pieces WHERE status = 'in_stock'\)](#)

Price resolution

```
lot.sale_price → variant.sale_price
```

[items.sale_price](#) is the template that seeds new variants; the variant is authoritative.

2. Tables built

2.1 item_variants

```

ulid id, ulid item_id
jsonb attributes          // {"ram":"16GB","color":"black"}
string variant_key        // canonical sorted fingerprint
string name, sku, barcode
decimal(18,4) sale_price, purchase_price, avg_cost, minimum_stock, maximum_stock, weight
integer sort_order        // S,M,L sort wrong alphabetically
boolean is_default, is_active
ulid branch_id + audit columns, timestamps, softDeletes

```

```

CREATE UNIQUE INDEX item_variants_identity_unique
  ON item_variants (item_id, variant_key) WHERE deleted_at IS NULL;
CREATE UNIQUE INDEX item_variants_barcode_unique
  ON item_variants (branch_id, barcode)
  WHERE deleted_at IS NULL AND barcode IS NOT NULL;
CREATE INDEX item_variants_attributes_gin ON item_variants USING GIN (attributes);

```

`variant_key` is derived on save, never supplied — sorted and lower-cased, so `{"ram":"16GB","color":"Black"}` and `{"color":"black","ram":"16gb"}` collide instead of splitting stock across two rows.

Trap: `attributes` is also Eloquent's internal storage property. Inside `ItemVariant`, `StockPiece` or `Item` it **must** be read with `getAttribute('attributes')`. Plain `$this->attributes` returns the whole column array and silently produces garbage — this shipped once and had to be repaired across 286 rows.

2.2 stock_lots

```

ulid id, ulid item_id, ulid variant_id (nullable)
string batch_no (nullable)    // null for milk
date expire_date (nullable)   // null for paint
date manufacture_date, received_date
decimal(18,4) mrp, sale_price, unit_cost, purchase_price
enum status                  // active|blocked|recalled|expired
ulid branch_id + audit columns, timestamps, softDeletes

```

```

CREATE UNIQUE INDEX stock_lots_identity_unique
  ON stock_lots (branch_id, item_id, variant_id, batch_no, expire_date)
  NULLS NOT DISTINCT WHERE deleted_at IS NULL;

```

`NULLS NOT DISTINCT` is **required** here (unlike on variants): two lots both with `batch_no` NULL and the same expiry are the same lot, so the NULLs must compare equal. Without it you silently get duplicate milk lots, each with its own price.

Expiry-only lots are the common case in food retail, not an edge case.

No `supplier_id` — the purchase line already reaches the supplier via `lot` → `purchase_item` → `purchase`.

2.3 stock_pieces

Named *pieces*, not *units*, to avoid colliding with unit-of-measure.

```

ulid id, item_id, variant_id, lot_id (nullable), warehouse_id
jsonb identifiers          // {"imei1":..., "imei2":..., "vin":..., "hallmark":...}
jsonb attributes           // {"length":2.1, "width":3.0, "net_weight":8.4}
decimal measured_quantity // 6.3 m² / 8.4 g

```

```

decimal unit_cost          // specific identification
decimal sale_price
enum condition, status
ulid purchase_item_id (nullable)
date warranty_end_date
ulid branch_id + audit columns, timestamps, softDeletes

```

`identifiers` is JSONB, not a `serial_no` column: dual-SIM phones need **two** IMEIs, vehicles need VIN and engine number, carpets need neither.

No `sale_item_id`. Indivisible pieces are sold once, but divisible ones (a 100 m cable drum sold 5 m at a time) are consumed by many sales, which a single foreign key cannot express. Traceability comes from `stock_movements.piece_id`.

2.4 items — fields added

Behaviour flags — a business type is a *combination of these*: `is_active`, `is_stockable`, `is_sellable`, `is_purchasable`, `is_serial_tracked`, `is_weighted`, `has_variants`, `allow_negative_stock`

Costing / pricing: `costing_method` (nullable → company default), `pricing_method`

Physical: `description`, `photo`, `weight`, `length`, `width`, `height`

Sourcing & compliance: `manufacturer`, `model`, `country_of_origin`, `hs_code`, `warranty_months`, `shelf_life_days`, `min_shelf_life_percent`, `storage_zone`, `requires_prescription`, `is_controlled`

Planning: `reorder_quantity`, `lead_time_days`, `default_warehouse_id`, `attributes` (descriptive, non variant-forming)

`is_batch_tracked` and `is_expiry_tracked` stay **two independent booleans**. They are not collapsible into one enum — most supermarket perishables are expiry-tracked with no batch code at all.

2.5 Ledger and line tables

`stock_balances` → `variant_id`, `lot_id` `stock_movements` → `variant_id`, `lot_id`, `piece_id` 12 transaction line tables → `variant_id`, `lot_id`, `piece_id`

`stock_balances` deliberately has **no `piece_id`**: it is an aggregate, and a piece is always quantity 1 — adding it would turn 500 phones into 500 balance rows duplicating the 500 piece rows.

3. Business-type separation

`config/business_profiles.php` → `App\Support\BusinessProfile` → shared to Inertia as `business_profile` → `useBusinessProfile()` in Vue.

Three layers, lowest first:

1. `base` — sensible defaults for all 51 toggleable fields
2. `profiles.<business_type>` — what this trade differs on
3. `company.preferences.item_management` — what the owner overrode

Every toggleable field must appear in `base.fields`. A field missing from that map can never be shown, because the forms read visibility from it.

Measured coverage

Business	Spec	Variants	Lots	Pieces	Fields on
pharmacy	batch	yes	yes	—	32 / 51
supermarket	expiry	—	yes	—	30 / 51
grocery	expiry	—	yes	—	25 / 51
manufacturing	lot	yes	yes	—	33 / 51
retail	batch	yes	—	—	24 / 51
accounting	batch	—	—	—	14 / 51
automobile	batch	yes	—	yes	30 / 51
book	batch	yes	—	—	23 / 51
clothing	batch	yes	—	—	27 / 51
hotel	batch	—	—	—	22 / 51
carpet	batch	yes	—	yes	28 / 51
electronics	batch	yes	—	yes	27 / 51
computer	batch	yes	—	yes	26 / 51
jewellery	batch	yes	—	yes	28 / 51
mobile	batch	yes	—	yes	27 / 51
pharma_manufacturing	batch	yes	yes	—	34 / 51
ply_carpet	batch	yes	—	yes	25 / 51

37 of 51 fields vary by trade; 14 are always on.

Adding a business type is a key in this config plus enum cases. Never a migration.

4. Enums

Enum	Change
<code>CostingMethod</code>	added <code>SPECIFIC</code> (jewellery, carpet, vehicles)
<code>BusinessType</code>	added <code>GROCERY</code>
<code>LotStatus</code>	new — active / blocked / recalled / expired
<code>PieceStatus</code>	new — in_stock / reserved / sold / returned / scrapped
<code>ItemCondition</code>	new — new / refurbished / used / damaged
<code>PricingMethod</code>	new — fixed / weight / area / length based

`costing_method` is now per-item, falling back to the company setting. A jewellery shop sells gold (specific) *and* gift boxes (weighted average).

5. Cross-cutting fixes applied

PostgreSQL NULL uniqueness

Every table using `unique([...], 'deleted_at')` had a **non-functioning constraint** — PostgreSQL treats NULLs as distinct and `deleted_at` is NULL on every live row, so duplicate names and codes were possible on `items`, `categories`, `brands`, `warehouses`, `sizes` and `unit_measures`. The matching validation rules used `unique:items,name,NULL,id,branch_id,NULL,...`, which tests `branch_id IS NULL` and never matched either — so duplicates escaped **both** layers.

Replaced with partial unique indexes over live rows, and properly scoped `Rule::unique` in the requests.

Other defects closed

- `destroy()` returned a redirect **inside** a `DB::transaction` closure, so the value was discarded, `$item->delete()` never ran, and the user was told the item had been deleted.
- `ItemUpdateRequest` reused the store rule for `sku` with no `ignore()`, so an item conflicted with itself on save.
- `required_with:sale_price<0` and `required_with:openings.*.quantity>0` are not valid rule expressions and never fired.
- `ItemListResource` ran a `SUM` per row for on-hand; now `withSum` + `withCount`.
- The index declared Brand and Sale Price columns the resource never sent.
- `items.photo` was in `$fillable` and assigned in `update()` but the column did not exist.
- `restore` and `forceDelete` had no authorization — `authorizeResource` covers only the seven RESTful actions.
- `categories.is_active` was cast and fillable with no column.

6. Still outstanding

Deferred by decision, in rough priority order:

Item	Note
Lots / pieces UI	Tables and models exist; rows are created by stock transactions, not the item form
Attribute definition tables	<code>attributes</code> , <code>attribute_values</code> , <code>category_attributes</code> . The <code>attributes</code> JSONB column already exists, so this is UI plus validation only
<code>item_barcodes</code>	Needed once a trade wants both carton and unit codes — routine for a supermarket
<code>market_rates</code> + computed pricing	Jewellery and carpet cannot function without it. <code>pricing_method</code> is reserved on <code>items</code> so it drops in cleanly
<code>tax_rates</code>	Leaf table plus line columns; costs no more later than now
<code>item_suppliers</code> , <code>item_relations</code> , <code>item_components</code> , <code>item_fitments</code>	Derivable or deferred
<code>price_lists</code>	<code>rate_a/b/c</code> are the stopgap
Manufacturing module	BOM, production orders, WIP, routing

Known pre-existing failures

`php artisan test --testsuite=Feature` reports **9 failures on a clean checkout** of this branch, in `StockAdjustmentFeatureTest`, `ItemTransferFeatureTest`, `SaleFeatureTest`, `ReportsFeatureTest` and `NotificationPreferencesFeatureTest`. They predate this work — verified by stashing and re-running. The stock

ones fail with "Insufficient stock: available 0", which points at the stock engine rather than the item module.

7. What already worked — do not change

- **FEFO allocation.** `StockService::deductFIFO` orders by `expire_date` with nulls last, then date. Correct for supermarket and pharmacy, and it already handles the mixed expiry/non-expiry case.
- **Reservations.** `reserved_out` / `reserved_in` prevent overselling from unposted lines.
- `spec_text` in preferences already relabels the lot identifier per trade.
- **Polymorphic attachments** — extend with `collection` and `is_primary` for image galleries rather than adding more columns.